



Algoritmi Ricorsivi

Ripasso Algoritmi Ricursivi, Ricorsione di Testa vs Coda

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Richiami su Algoritmi Ricorsivi

Algoritmo Ricorsivo



- Procedimento che richiama se stesso per fare il calcolo
- Esempio: definizione del fattoriale
 - Dato un numero naturale $n \geq 0$, il suo fattoriale $n!$ è calcolato come segue:

$$\begin{array}{ll} n! = 1 & \text{se } n \text{ vale } 0 \text{ oppure } 1 \\ n! = n * (n-1)! & \text{se } n > 0 \end{array}$$

- Quant'è $5!$?

$$\begin{aligned} 5! &= 5 * 4! \\ &= 5 * (4 * 3!) \\ &= 5 * 4 * (3 * 2!) \\ &= 5 * 4 * 3 * (2 * 1!) \\ &= 5 * 4 * 3 * 2 * 1 \end{aligned}$$



$n!$ è calcolato come n per il fattoriale del numero che precede n stesso

Fattoriale: in C



$n!$ è calcolato come n per il fattoriale del numero che precede n stesso

- Elementi caratteristici:

```
unsigned int fattoriale(unsigned int n) {  
    if (n <= 1) {  
        return 1;  
    } else {  
        return n * fattoriale(n-1);  
    }  
}
```

Caso base: definisce il punto di stop

Chiamata ricorsiva: invocazione di se stessa

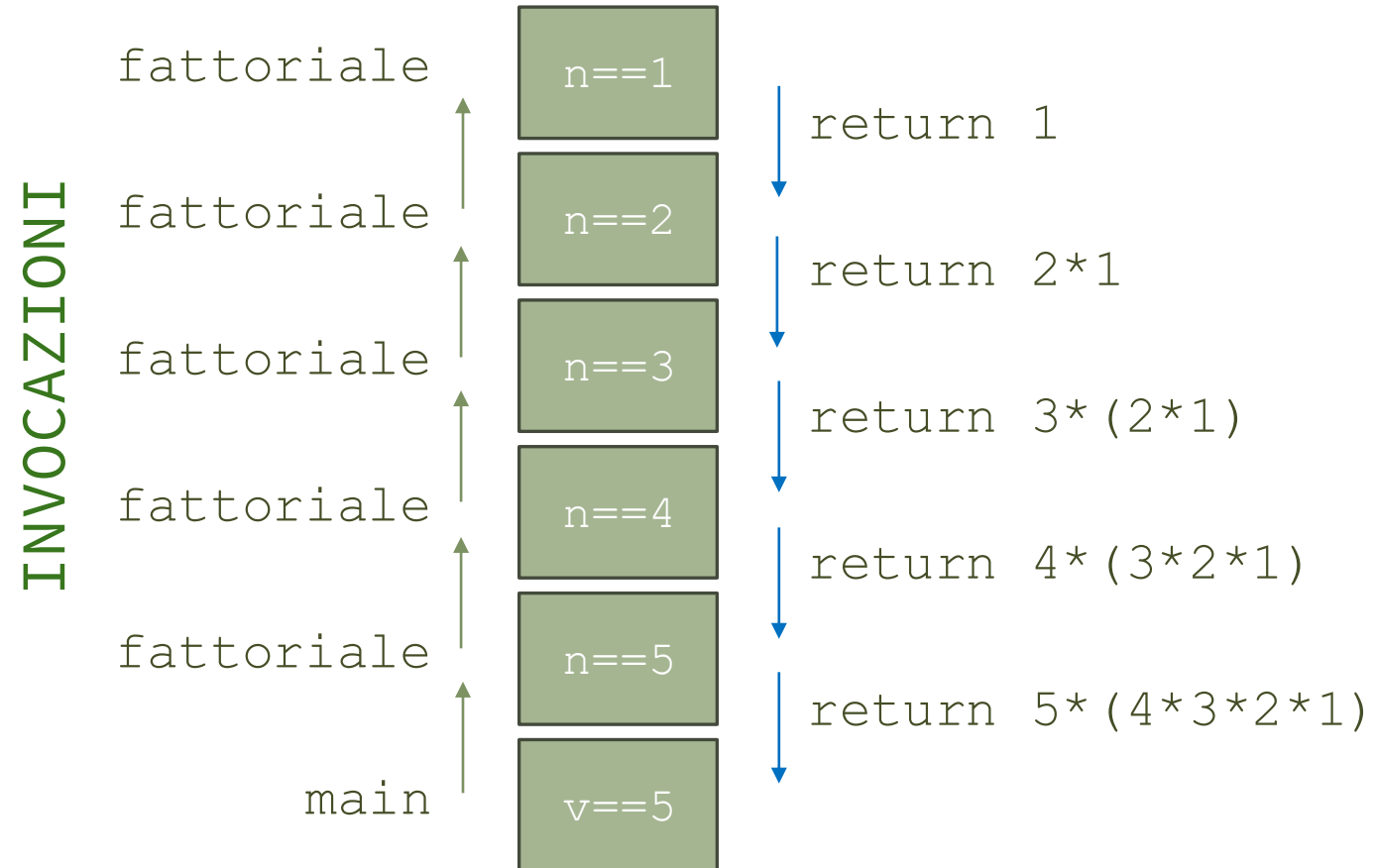
Progresso: la chiamata ricorsiva elabora altri dati, non gli stessi della chiamata attuale

Stack di Esecuzione



- La ricorsione è detta **lineare** quando produce una singola sequenza di chiamata
 - Esempio: chiamata a `fattoriale(5)` dal `main`

```
uint32_t fattoriale(uint32_t n) {  
    if (n <= 1) {  
        return 1;  
    } else {  
        return n * fattoriale(n-1);  
    }  
}  
  
int main(void) {  
    uint32_t v = 5;  
    ...fattoriale(v);  
}
```



Assenza di Progresso

```
unsigned int fattoriale(unsigned int n) {  
    if (n <= 1) {  
        return 1;  
    } else {  
        return n * fattoriale(n);  
    }  
}
```

- Non aggiornare i dati nella chiamata ricorsiva:
 - Errore Grave
 - Perché?
 - Esecuzione infinita
- Una funzione ricorsiva deve sempre terminare
 - Occorre essere sicuri che a un certo punto sarà raggiunto il caso base



Commenti

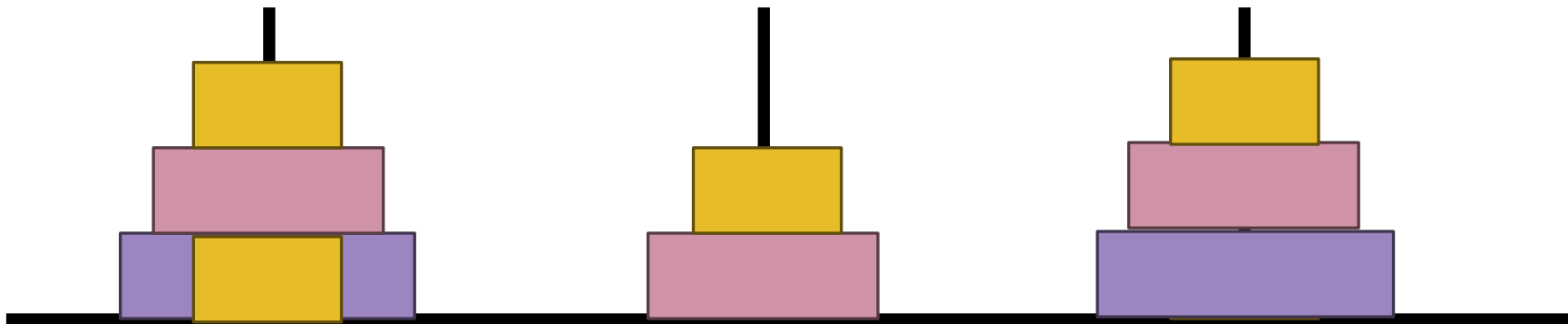


- La ricorsione (spesso) permette di scrivere **codice compatto e facile da interpretare**
 - Rispetto ad un'implementazione iterativa
 - ... ma non sempre

Torri di Hanoi



- Classico puzzle matematico con tre pali e N dischi
 - Obiettivo: spostare tutti i dischi da A a B senza mai mettere un disco più grande su uno più piccolo



Torri di Hanoi: Soluzione Ricorsiva Compatta



- Implementazione ricorsiva

```
int towerOfHanoi(int n, char src, char dst, char aux, int m) {  
    if (n == 0) {  
        return m;  
    }  
    m = towerOfHanoi(n - 1, src, aux, dst, m); m++;  
    printf("Mossa %d: Disco %d spostato da %c -> %c\n",  
          m, n, src, dst);  
    m = towerOfHanoi(n - 1, aux, dst, src, m);  
    return m;  
}
```

Caso base

Chiamata ricorsiva

```
int main(void) {  
    int n = 0;  
    printf("\nNumeri di dischi da usare: "); // Leggo il numero di dischi  
    scanf("%d", &n);  
    towerOfHanoi(n, 'A', 'C', 'B', 0); // A, B and C sono i nomi dei pali  
}
```

Torri di Hanoi: Soluzione Iterattiva



- Implementazione iterattiva

 Molto più complessa

```
typedef struct pole {
    int *pole;
    int n;
    char name;
} Pole;

void move(Pole *X, Pole *Y) {
    int d = X->pole[X->n-1];
    // Sposto disco
    Y->pole[Y->n] = d;
    Y->n++;

    // Rimuovo disco
    X->n--;
    X->pole[X->n] = 0;

    printf("Disco %d spostato da %c -> %c\n", d, X->name, Y->name);
}

void movement(Pole *X, Pole *Y, int s) { // Assume s = {0,1}
    if (s == 1) {
        if(X->pole[X->n-1] == 1) {
            move(X, Y);
        } else if (Y->pole[Y->n-1] == 1){
            move(Y, X);
        }
    } else {
        if (Y->n != 0 && X->pole[X->n-1] > Y->pole[Y->n-1]) {
            move(Y, X);
        } else if (Y->n == 0) {
            move(X, Y);
        } else if (X->n != 0 && Y->pole[Y->n-1] > X->pole[X->n-1]) {
            move(X, Y);
        } else if (X->n == 0) {
            move(Y, X);
        }
    }
}
```

```
void display_stack(Pole *p) {
    if (p->n == 0) {
        printf("\n%c è vuoto\n", p->name);
    } else {
        printf("\n%c contiene (%d) :\n", p->name, p->n);
        for (int i = p->n-1; i>=0; --i) {
            printf("  %d\n", p->pole[i]);
        }
    }
}

void display(Pole *A, Pole *B, Pole *C) {
    display_stack(A); display_stack(B); display_stack(C);
}

int main() {
    Pole A = {NULL, 0, 'A'}, B = {NULL, 0, 'B'}, C = {NULL, 0, 'C'}; int n=0, max=1, step=0;

    printf("\nNumeri di dischi da usare: "); scanf("%d",&n); // Leggo il numero di dischi
    A.pole = malloc(n * sizeof(*(A.pole))); B.pole = malloc(n * sizeof(*(B.pole)));
    C.pole = malloc(n * sizeof(*(C.pole)));

    for (int i=0; i<n; i++) { max = max*2; } // Calcolo del numero di mosse necessarie
    max = max-1;

    for (int i=0; i<n; i++) { A.pole[i] = n-i; } // Aggiungo n dischi al primo palo
    A.n = n;

    // Mostro situazione iniziale
    printf("\nStato iniziale:"); display(&A, &B, &C);

    for(int c=1; c <= max && C.n != n; c++) {
        printf("Mossa %d/%d: ", c, max); step = (step%3) + 1;
        if(step == 1) movement(&A, &C, c%2);
        else if (step == 2) { movement(&A, &B, c%2);
        else if (step == 3) { movement(&C, &B, c%2);
        }
        printf("\nStato finale:"); display(&A, &B, &C);
    }
```

Controesempio: Successione di Fibonacci

- La successione di Fibonacci

- L'ennesimo numero di Fibonacci f_n è la somma dei due numeri precedenti: $f_n = f_{n-1} + f_{n-2}$
 - Iniziando con $f_1=1$ e $f_2=1$

- Soluzione iterativa

- Compatta e efficiente

```
uint32_t fibonacciIterative(uint32_t N) {  
    uint32_t n = 0, n1 = 1, n2 = 1;  
    for (uint32_t i = 0; i < N; i++) {  
        n = n1;  
        n1 = n2;  
        n2 = n + n1;  
    }  
    return n;  
}
```

- Soluzione ricorsiva

- Problema?

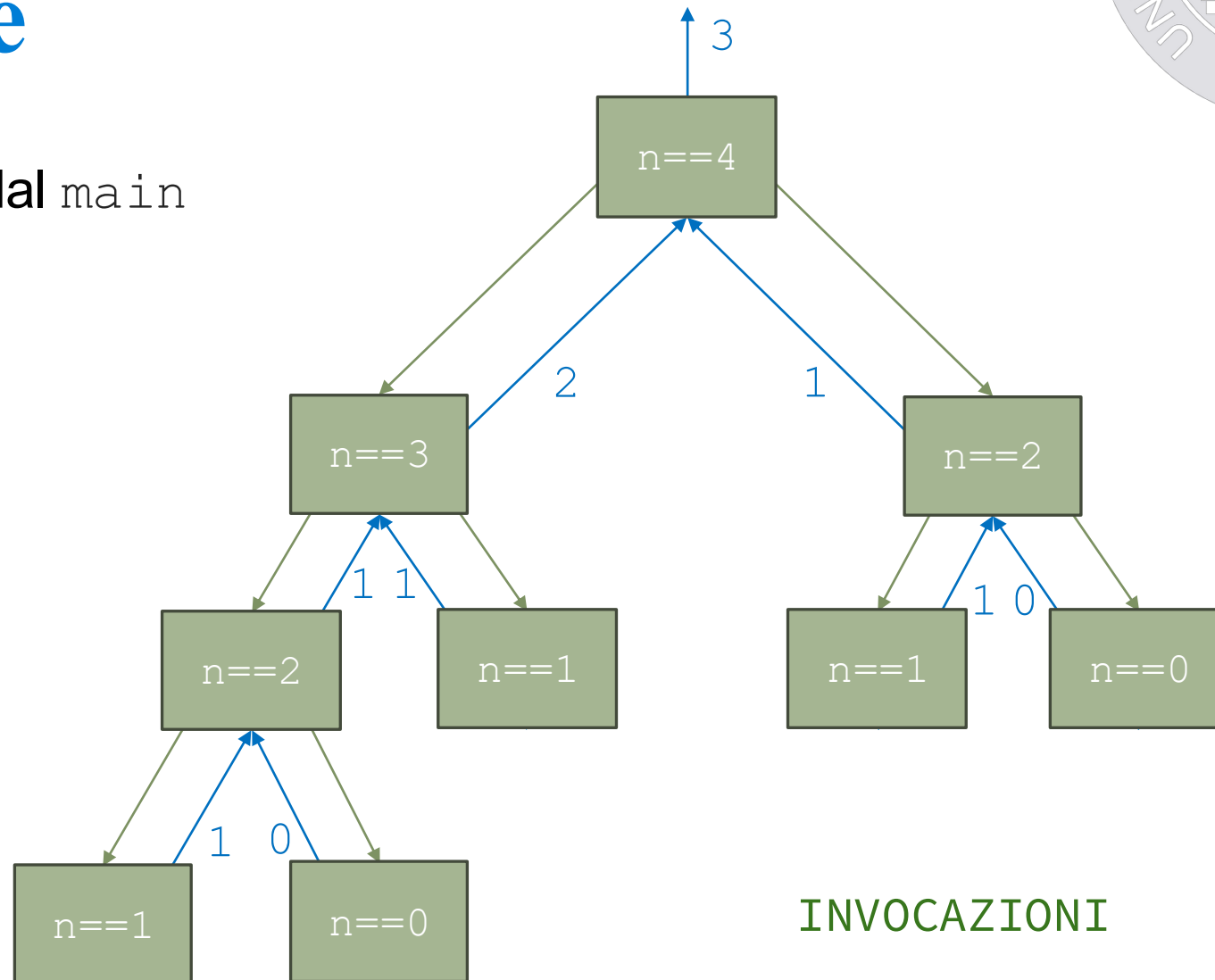
```
uint32_t fibonacciRicorsive(uint32_t n) {  
    if ((n < 2)) {  
        return n;  
    }  
    return fibonacciRicorsive(n - 1)  
        + fibonacciRicorsive(n - 2);  
}
```

Complexità quadratica: due chiamate ricorsive

Stack di Esecuzione

- Esempio: chiamata a `fibRec(4)` dal `main`
 - Invocazione creano un albero

```
uint32_t fibRic(uint32_t n) {  
    if (n < 2) {  
        return n;  
    }  
    return fibRic(n - 1)  
        + fibRic(n - 2);  
}
```



INVOCAZIONI

return



Commenti



- La ricorsione **comporta un overhead**
 - Tempo extra di esecuzione e occupazione di memoria
 - Dovuto alla creazione/rimozione del relativo record di attivazione di ogni chiamata di funzione

Ricorsione di Coda Vs Testa



- Ricorsione di Coda

- Dopo la chiamata ricorsiva non si esegue altro codice

```
uint32_t fattoriale(uint32_t n) {  
    if (n <= 1) {  
        return 1;  
    } else {  
        return n * fattoriale(n-1);  
    }  
}
```

Ricorsione di coda: ultima istruzione prima di `return`

Caso base: raggiunto dopo una serie di chiamate ricorsive

- Ricorsione di Testa

- Ricorsione come prima operazione, seguita da altre
- In genere necessita di una variabile per valori temporanei

```
uint32_t fattoriale(uint32_t n) {  
    unsigned int temp;  
    if (n > 1) {  
        temp = fattoriale(n-1);  
    } else {  
        temp = 1;  
    }  
    return n*temp;  
}
```

Variabile: per valori temporanei

Ricorsione di testa: è seguita da altre operazioni

Restituzione: ulteriore calcolo dopo la chiamata ricorsiva

Chiamata di Coda o Terminale



Definizione

In una funzione f_1 , una chiamata ad una funzione f_2 si dice chiamata di coda se in f_1 , dopo la chiamata di f_2 non ci sono altre istruzioni da eseguire.

- Chiamate in coda possono essere **ottimizzate** dal compilatore
 - Consiste nell'evitare di allocare un record di attivazione per la funzione f_2
 - Fattibile se la funzione f_1 vede solamente il valore restituito da f_2
 - Questa ottimizzazione è conveniente nel caso di «sibling call», ovvero, quando f_2 ha:
 - Parametri e valore di ritorno che occupano lo stesso spazio occupato da quello di f_1

Dettagli nella documentazione del [compilatore](#).

Chiamate di Coda: Esempi

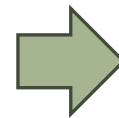
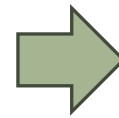
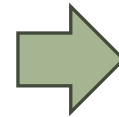


- Chiamate NON di coda

```
int f(int x 1,...,int x n) {  
    ...  
    return 1 + g(y 1,...,y k);  
}
```

```
void f(int x 1,...,int x n) {  
    ...  
    g(y 1,...,y k);  
    printf("`Cioa!`");  
}
```

```
int f(int x 1,...,int x n) {  
    ...  
    if (y 1 > y 2) {... g(y 1,...,y k); return y;}  
    else {... h(z 1,...,z l); return z; }  
}
```



- ... trasformate in chiamate di coda

```
int f(int x 1,...,int x n) {  
    ...  
    return g(y 1,...,y k);  
}
```

```
void f(int x 1,...,int x n) {  
    ...  
    g(y 1,...,y k);  
}
```

```
int f(int x 1,...,int x n) {  
    ...  
    if (y 1 > y 2) {... return g(y 1,...,y k); }  
    else {... return h(z 1,...,z l); }  
}
```


Ricorsione di Coda



- Chiamate **ricorsive di coda**
 - Sono «sibling call»
 - Riducono la complessità spaziale delle chiamate a funzione ricorsiva
- È quindi importante, quando possibile, scrivere le funzioni ricorsive in modo che siano ricorsive di coda.
 - Come?

Tecnica per Funzioni Ricorsive di Coda



- Una tecnica per trasformare una funzione ricorsiva NON di coda in una funzione ricorsiva di coda è:
 - Aggiungere un parametro “accumulatore” che accumula il risultato
- Chiamate NON di coda
- ... trasformate in chiamate di coda

```
int fact(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fact(n-1);  
}
```

```
int factAux(int n, int acc) {  
    if (n <= 1)  
        return acc;  
    else  
        return factAux(n-1, acc * n);  
}  
  
// Funzione wrapper (non ricorsiva)  
int fact(int n) {  
    return factAux(n, 1);  
}
```

Accumulatore